CC Expt. 5

Name: Tanay Manish Nesari

UID: 23BCS13761

Class: 2-B

Q1) Watto recently got an order for mechanism that can process strings in a certain way. Initially, memory of mechanism has n strings. Then mechanism should be able to process queries of type: "given string s, determine if memory of mechanism has string t that has same no. of characters as s and differs from s at exactly 1 position."

Watto has already compile the mechanism, all that is left is to write a code for it and check it on data consisting of n lines & m queries".

Input: 2 3

aaaaa

acacaca

aabaa

ccacacc

caac

Output: YES

NO

NO

Algorithm / Approach:

1. Take int n and int m as input
2. take input of n strings in memory
3. take input of m queries
4. Create an empty hashset patterns and insert the n strings into it
5. ~~For each of the n stored strings, replace character at index i for every position i in string with *, store modified string in patterns~~
6. ~~For each of the m query strings, initialize found = false~~
7. For every position in query string, replace the character with other possible characters.
8. If, after replacement the modified string exists in patterns then print "YES", if no such modified string exists in patterns then print "NO".

Code:

```
import java.awt.*;
import java.io.*;
import java.util.*;
Public class Main {
    Static int Mo = (int)1e9 + 9;
    Static long[] Pow;
    Static long P = 11;
    Public static void main (String[] args) throws Exception {
        Scanner Sc = new Scanner (System.in);
        StringBuilder Sb = new StringBuilder();
        int n = Sc.nextInt(), m = Sc.nextInt();
        Set<Long> hs = new HashSet<>();
        Pow = new Long[1000 * 1000 + 1];
        Pow[0] = 1
        for (int i=1; i < Pow.length; i++) {
            Pow[i] = (P * Pow[i-1]) % Mo;
        }
        for (int i=0; i<n; i++) {
            hs.add (ComputeHash (Sc.next().toCharArray()));
        }
        for (int i=0; i<m; i++) {
            Char[] temp = Sc.next().toCharArray();
            long val = ComputeHash (temp);
            Boolean z = false;
```

```
for (int j=0; j< temp.length && !z; j++){
        char x = temp[j];
        long res = (x * Pow[j])% Mo;
        Val -= res;
        for (char c='a'; c<='c' && !z; c++){
            if (c!=x){
                long newRes = (c * Pow[j])% Mo;
                Val += newRes;
                z= hs.contains (Val);
                Val -= newRes;
            }
        }
        Val += Res;
    }
    If(z)Sb.append ("YES \n");
    else Sb.append ("No \n");
}
System.out.println (Sb);
}
Static long ComputeHash (char[] c){
    long hashvalue = 0;
    for (int i=0; i< c.length; i++){ hashvalue += (c[i] * Pow[i])% Mo;
    return hashvalue;
}
}
```